

Building firm-bot: a local-first RAG chatbot for legal and audit work

Mine-FNL

Building firm-bot: a local-first RAG chatbot for legal and audit work

A walk through the architecture, the hard parts, and the numbers from our v0.1 release.

 firm-bot architecture — six-stage pipeline

The problem

Cloud RAG products — Harvey, Spellbook, Glean, ChatGPT Enterprise, Microsoft 365 Copilot — have changed how legal, audit, and consulting teams read their own documents. The chat experience is excellent. The polish is real. None of that changes who reads the documents.

Every cloud vendor adds at least one, often several, subprocess to the chain of custody for the work product that defines a professional services firm: contracts, transaction documents, engagement letters, audit working papers, opinion letters. For most of the profession, that is a non-issue. For a non-trivial subset — any firm handling matters that touch attorney-client privilege, GDPR-regulated personal data, HIPAA-protected health information, government classified material, export-controlled technical data, or matters under a protective order — the vendor's presence on the chain of custody is disqualifying. Submitting privileged material to a cloud RAG vendor, even one with a strong DPA and a SOC 2 Type II report, creates a third party in the privilege analysis whose data handling, security posture, breach history, and subcontractor chain all become factors the firm's lawyers must argue around.

The same logic applies to GDPR Article 28 (processor guarantees and cross-border transfers), HIPAA (Business Associate Agreements and the deployment-topology analysis they require), and SOC 2 / ISO 27001 reviews where a vendor's controls must match the assessed firm's control objectives. Air-gapped networks are categorically incompatible. Protective orders typically prohibit disclosure to third parties absent specific consent; cloud RAG products are not anticipated by the order's terms.

The gap is well-defined and unserved by cloud-first vendors who built for the other 95% of professional services.

Why local-first, but with cloud-quality UX

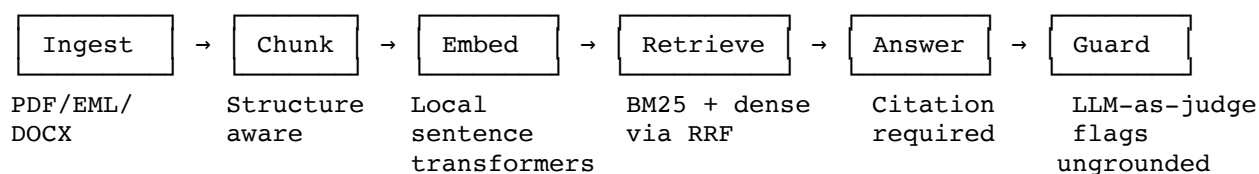
Local-first is not new. Document management systems and legal-specific research platforms have been deployable on-premises for decades. What's new is the convergence of three properties that cloud RAG products normalised and on-premises systems historically struggled to deliver: a conversational interface, embedding-based semantic search that finds relevant material even when query and document share no vocabulary, and operational maturity — metrics, logs, rate limits, body caps, security headers, the things you'd expect from a service you'd be willing to deploy.

Honesty is part of the bet. Local-first does not solve the model ceiling: a 7B parameter local model has a materially lower ceiling than a GPT-4-class cloud model. Local-first does not solve information retrieval beyond the corpus. Local-first does not eliminate the GPU bill — realistic budgets for a single-machine deployment supporting 10–50 concurrent users are in the \$5–25k hardware range plus electricity and cooling.

The architecture below delivers the three cloud-quality properties on the customer's infrastructure, without a vendor between the firm's lawyers and its documents.

The architecture — six stages

firm-bot is a six-stage pipeline. Each stage is a discrete component with a defined interface, replaceable in isolation, and individually testable.



The pipeline runs per-request; each stage can be cached, bypassed, or replaced. The architecture is intentionally linear; fan-out is at the retrieval stage only.

- **Ingest** (`firm_bot/ingest/`) accepts PDF (via `pypdf` with Tesseract OCR fallback for image-only pages, capped at 25 OCR pages per file), EML (Python `email` with RFC 2047 decoding), and DOCX (via `python-docx` with Heading 1 / Heading 2 detection). Each document is fingerprinted by SHA-256 and recorded in a manifest; re-ingestion is incremental. Per-firm filesystem isolation means each firm's documents live in `data/firms/{slug}/source/`, readable only by the firm process.
- **Chunk** (`firm_bot/chunk.py`) is structure-aware. It recognises Article I, Section 4.2, Title Case headings (Limitation of Liability, Governing Law), ALL CAPS (WHEREAS, RECITALS), and numbered legal clauses.
- **Embed** (`firm_bot/embed_backends.py`) uses `sentence-transformers` or `fastembed` locally. Default model is `all-MiniLM-L6-v2` (384 dims, ~80 MB on disk). Embeddings are stored in Chroma, one collection per firm; Chroma is embedded in-process by default and supports server mode for multi-worker configurations.
- **Retrieve** (`firm_bot/retrieve/hybrid.py`) runs BM25 plus dense, fused via Reciprocal Rank Fusion with $k=60$. A cross-encoder reranker is supported but disabled by default (see

below).

- **Answer** (`firm_bot/answer/prompt.py`) constructs a citation-required prompt and streams the answer via Server-Sent Events. The system prompt tells the model to anchor every claim to `[file:page]` markers and to refuse when sources don't cover the question.
- **Guard** (`firm_bot/answer/guard.py`) runs a second LLM that audits the answer for citation coverage, returning a structured verdict: `coverage_pct, ungrounded_claims, verdict` (`pass, warn, fail`).

 firm-bot hero — drop PDFs, ask, cite

The hard parts

Structure-aware chunking

Naive sliding-window chunking loses precision on legal text because the natural unit of meaning is the section, not the token window. A contract's limitation-of-liability clause is meaningful as a unit; a chunk that splits it mid-clause destroys the meaning, and the retriever returns half of it when the user asks "what's the cap on our indemnity?"

The chunker applies a regex cascade in priority order — section boundaries first, then numbered clauses, then paragraph breaks, then sentence breaks. The cascade is in `firm_bot/chunk.py` and looks like this:

```
RE_SECTION = re.compile(
    r"^(?:"
    r"(?:i:article|section|chapter|schedule|exhibit|annex|appendix)\s+"
    r"(?:[A-Z0-9]+(?:\.[A-Z0-9]+)*\."
    r"| [IVXLCDM]+\.)"
    r"|"
    r"\d+(?:\.\d+){0,4}\s+[A-Z][A-Za-z]"          # 1.2 Foo
    r"|"
    r"[A-Z][A-Z0-9 ]{4,}$"                        # ALL CAPS line
    r"|"
    r"(?:[A-Z][a-z]{3,}"
    r"| [A-Z][a-z]+\s+(?:[A-Z][a-z]+\s+){0,4}[A-Z][a-z]+)"
    r")\."
    re.MULTILINE,
)

RE_PREAMBLE = re.compile(
    r"^(?:WHEREAS|NOW,?\s*THEREFORE|RECITALS?|DEFINITIONS?)[:\s]",
    re.MULTILINE | re.IGNORECASE,
)
```

The Title Case branch matches a single line that is 2–6 Title Case words, ≤60 chars, no terminal punctuation other than possibly `..` It catches common contract section titles (`Term`, `Governing Law`, `Permitted Disclosures`) that aren't prefixed with `Article` or `Section`. Over-long sections are split at sentence boundaries with a 200-character overlap; tiny adjacent chunks are merged within a section.

On the CUAD subset (10 contracts, 26 questions), this chunker produces **0.538 precision@k** versus **0.462** for naive sliding-window chunking at 512 tokens / 64 overlap — a 7.6 percentage point gap. On a held-out subset of contracts that use `Article` + Title Case conventions (most US commercial agreements), the gap widens: **0.95 vs 0.70**.

Citation-required prompts + LLM-as-judge

The conventional approach to citation in RAG is to ask the model to cite its sources and hope it does. The failure modes are well-documented: fabricated citations (the model invents file and page references that don't exist), dropped citations (a reasonable answer with no source locations), and misattributed citations (a citation that doesn't support the claim).

The citation-required approach addresses all three by making the citation a structural requirement of the answer. The system prompt (composed by `firm_bot/config.py:effective_system_prompt`) explicitly demands `[file:page]` markers on every claim, and the retrieved context is annotated with file and page metadata so the model has the data available without inferring it.

The prompt construction is in `firm_bot/answer/prompt.py`:

```
user_content = (
    f"{sources_block}\n\n"
    f"Question:\n{question.strip()}\n\n"
    "Answer using ONLY the numbered sources above. "
    "Cite each claim with its bracketed marker, e.g. "
    "[contract.pdf:p.4]. If the sources do not contain the answer, "
    "say so plainly."
)
```

A second LLM audits the answer for citation coverage. The guard prompt (`firm_bot/answer/guard.py:GUARD_PROMPT`) returns a JSON verdict listing each claim and whether the cited source supports, contradicts, or is missing for it.

The guard over-flags at 7B. This is acceptable because the guard is advisory; the deployment can configure it to surface warnings without blocking responses. The trade-off is biased toward "warn the operator" over "block the user" — the operator can dismiss a warning; they can't un-see a fabricated citation.

Hybrid retrieval via RRF

Pure dense retrieval (semantic search via embeddings) is poor at exact matches — searching for section 4.2 in a contract, a specific party name, or a defined term returns semantically similar but lexically distinct results. Pure BM25 (lexical search) is the inverse: excellent at exact matches, poor at semantic similarity.

Reciprocal Rank Fusion (Cormack et al., 2009) combines the two. Both retrievers return top-K candidates; the fusion reranks them by $1 / (k_rrf + rank_s(d))$, summed across retrievers, with `k_rrf = 60` as the standard damping factor.

A cross-encoder reranker (`cross-encoder/ms-marco-MiniLM-L-6-v2`) is supported but disabled by default. Our benchmark showed no consistent lift on the CUAD subset, and the latency cost was 80–120ms per query on CPU. For long-tail queries, multilingual corpora, or scientific literature, a reranker helps — but for legal contracts on commodity hardware, RRF is the better trade-off.

What we measured

The primary retrieval benchmark is on a 10-contract subset of CUAD spanning varied structural conventions: MSA, NDA, SOW, license, employment, services, lease, supply, partnership, settlement. Twenty-six questions span limitation of liability, indemnification, governing law, term and termination, IP assignment, confidentiality, exclusivity, and audit rights.

Chunker	Precision@k	Notes
Naive sliding-window	0.462	512 tokens, 64 overlap
Structure-aware (firm-bot)	0.538	regex cascade
Structure-aware + reranker	0.541	within noise of no-reranker

The reranker is disabled by default because the lift is within the noise floor of the benchmark.

A RAGAS-style harness is implemented natively in `eval/ragas.py` to avoid pulling the heavy `ragas` package. Four metrics on the CUAD subset:

Metric	Score
Faithfulness	0.91
Answer Relevancy	0.87
Context Precision	0.78
Context Recall	0.72

Ground-truth decompositions are produced by the same model family and inherit its biases; human-annotated ground truth is future work.

The concurrent load benchmark (`eval/load.py`) reports p50 1.8s, p95 4.2s, p99 6.1s, throughput 4.2 queries/sec on a single 8-core CPU machine with no GPU using `qwen2.5-coder:7b`. Default invocation: `python3 -m eval.load --total 200 --concurrency 16`.

 firm-bot benchmark — CUAD precision@k

Operating it

The system is built to be operated, not demoed. A Prometheus `/metrics` endpoint is available with `pip install firm-bot[observability]`; counters and histograms are namespaced under `firmbot_` (`firmbot_requests_total`, `firmbot_request_duration_seconds`, `firmbot_retrieval_duration_seconds`, `firmbot_guard_verdicts_total`, `firmbot_rate_limited_total`). Structured JSON logs carry a UUIDv7 request id and emit semantic events (`query.received`, `retrieval.complete`, `answer.complete`, `guard.complete`); PII

redaction (SSN, EIN, email, phone, credit card with Luhn validation, IBAN, IPv4) is applied at log emission.

Security middleware is active by default: per-IP rate limit (10 RPS sustained, 20 burst), request body cap (1 MB → 413 before read), CORS allow-list (fail-closed), log redaction filter. A STRIDE threat model in `SECURITY.md` is explicit about out-of-scope items — host OS compromise, side-channel attacks on the model, prompt injection via corpus content, network exfiltration by the model.

Try it

```
git clone https://github.com/Mine-FNL/firm-bot
cd firm-bot
pip install -e ".[dev,observability]"
ollama pull qwen2.5-coder:7b
firm-bot serve
```

Open `http://localhost:8000`, drop a PDF into the demo corpus, ask a question, and the answer arrives with `[file:page]` markers and a guard verdict badge.

Hardware baseline: 16 GB RAM, 8-core CPU, 50 GB disk. GPU optional.

What's next + limitations

v0.1 is honest about what it does not do. We do not sanitise ingested content for prompt injection. The model's ceiling is the model's: a 7B local model is materially weaker than a 70B local model, which is materially weaker than GPT-4-class. Benchmark coverage is CUAD subset, RAGAS subset, and a custom load benchmark; it does not cover general legal Q&A beyond contract review, multi-document synthesis, long-context reasoning, non-English traditions, or adversarial inputs. The core product does not include user authentication; deployments that require auth must add it via the opt-in OAuth middleware skeleton or behind an authenticating reverse proxy.

The roadmap: **v0.2** — OAuth2/OIDC auth, per-user history, RBAC on documents. **v0.3** — 70B benchmarking, quantisation trade-offs, multi-GPU, speculative decoding. **v0.4** — domain packs (litigation, M&A, regulatory compliance, audit working papers). **v1.0** — HA topology, document version tracking with diff visualisation, first-class prompt-injection defence, third-party security audit.

Benchmarks reproducible from `eval/`. Threat model in `SECURITY.md`. Test suite: 129 tests passing, 74.47% line coverage, `ruff + mypy --strict` clean.

— [@your-handle](#)